Advanced JAVA and J2EE – I UNIT

2 marks:

1. With the syntax write the purpose of ordinal() method.

The ordinal() method is used to obtain a value that indicates an enumeration constant's position in the list of constants.

Syntax: final int ordinal()

2. List two key characteristics of enumeration constants in Java.

• In Java, an enumeration defines a class type. By making enumerations into classes, the capabilities of the enumeration are greatly expanded. For example, in Java, an enumeration can have constructors, methods, and instance variables.

• Each enumeration constant is public, static and final by default

3. What is an enumeration? How enumeration can be created?

An enumerationis a list of named constants. An enumeration is created using the *enum* keyword. For example, here is a simple enumeration that lists various colours:

```
enum Colors { White, Black, Red, Green
        }
```

The identifiers White, Black., and so on, are called enumeration constants.

4. Differentiate values() and valueOf() methods in Java enumerations.

The values() method returns an array that contains a list of the enumeration constants. Its general forms are shown here:

public static enum-type [] values()

The valueOf() method returns the enumeration constant whose value corresponds to the string passed in *str*. Its general forms are shown here:

public static enum-type valueOf(String str)

5. Provide an example of how to use the values() method to iterate through all the constants in an enumeration.

```
enum Colors {
      White, Black, Red, Green
}
class Demo {
      public static void main(String args[])
      {
            System.out.println("Here are all Color constants:");
            Colors colors[] = Colors.values();
            for(Colors c : colors) {
                  System.out.println(c);
                  System.out.println();
            }
      }
}
Output:     Here are all Color constants:
                  White
                  Black
                  Red
                  Green
```

6. Give the functionalities of compareTo(), and equals() methods in Java enumerations.

compareTo() compares the ordinal value of two constants of the same enumeration. It returns 0 if they are equal, returns a negative integer if the current enum constant comes before the specified enum constant and returns a positive integer if the current enum constant comes after the specified enum constant.

equals() checks if two enum constants are equal. It returns true if they are equal; otherwise returns false.

7. Why are type wrappers used in Java?

Type wrappers in Java, also known as wrapper classes, are used to encapsulate primitive data types into objects. Each primitive data type (such as int, char, boolean, etc.) has a corresponding wrapper class (Integer, Character, Boolean, etc.). Objects are needed if we wish to modify the arguments passed into a method.

8. What is the purpose of the doubleValue() method in a numeric wrapper class?(similarly other methods).

doubleValue( ) returns the value of an object as a double, floatValue( ) returns the value as a float, and so on. These methods are implemented by each of the numeric type wrappers.

9. What is the benefit of autoboxing and auto-unboxing in Java?

Autoboxing is the process by which a primitive type is automatically encapsulated (boxed) into its equivalent type wrapper whenever an object of that type is needed. There is no need to explicitly construct an object.

Auto-unboxing is the process by which the value of a boxed object is automatically extracted (unboxed) from a type wrapper when its value is needed. There is no need to call a method such as intValue( ) or doubleValue( ).

10. What is retention policy? How to set retention policy? Give an example.
A retention policy determines at what point an annotation is discarded. A retention policy is specified for an annotation by using one of Java's built-in annotations: ***@Retention***. Its general form is shown here:

```
@Retention(retention-policy)
```

Eg:
```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
        String str();
        int val();
}
```

11. List any two retention policies with its purpose.
• SOURCE is retained only in the source file and is discarded during compilation.
• CLASS is stored in the class file during compilation. It is not available through the JVM during run time.

12. What is the purpose of setting default values to annotation member. Write the general form for setting default values.

You can give annotation members default values that will be used if no value is specified when the annotation is applied

```
type member() default value;
```

Here, value must be of a type compatible with type.

13. Define a. Marker Annotation b. Single Member Annotation.
A marker annotation is a special kind of annotation that contains no members. Its sole purpose is to mark a declaration. Thus, its presence as an annotation is sufficient. The best way to determine if a marker annotation is present is to use the method is Annotation Present(), which is a defined by the AnnotatedElement interface.

A single-member annotation contains only one member. It works like a normal annotation except that it allows a shorthand form of specifying the value of the member. You can simply specify the value for that member when the annotation is applied—you don't need to specify the name of the member. However, in order to use this shorthand, the name of the member must be value.

14. List any two built in annotations with its purpose.

• @Retention – It specifies the retention policy.
• @Documented – It is a marker interface that tells a tool that an annotation is to be documented.

15. Write any two restrictions on annotation.

• No annotation can inherit another.
• All methods declared by an annotation must be without parameters.

16. What is the primary purpose of annotations in Java code?

Annotations in Java are a form of metadata that provide data about a program but are not part of the program itself. Annotations have no direct effect on the operation of the code they annotate. They can be used to provide information to the compiler, to be processed by tools, or to be available at runtime through reflection.

17. How are annotations declared in Java?Give example.

```
Syntax:     @interface AnnotationName{
                datatype member_name() [(@default value)]
            }
Eg:         @interface MyAnno {
            String str();
            int val() default 0;
    }
```

18. How can you retrieve all annotations with the RUNTIME retention policy associated with an element in Java reflection? Give its syntax.

You can obtain all annotations that have RUNTIME retention that are associated with an item by calling getAnnotations() on that item.

Syntax: Annotation[ ] getAnnotations( )
It returns an array of the annotations. getAnnotations( ) can be called on objects of type Class, Method, Constructor, and Field.

19. Name any two commonly used built-in annotations and briefly describe their purpose?

Refer Q.No. 14

20. What are some key characteristics of a Java Bean?

• A Java Bean is a software component that has been designed to be reusable in a variety of different environments.
• There is no restriction on the capability of a Bean.
• It may perform a simple function, such as obtaining an inventory value, or a complex function, such as forecasting the performance of a stock portfolio.
• A Bean may be visible to an end user. A example of this is a button on a graphical user interface.

21. List two advantages of Java Beans.

• The java beans possess the property of “Write once and run anywhere”.
• Beans can work in different local platforms.
• The properties, events and methods of the bean can be controlled by the application developer.
• The configuration setting can be made persistent. (reused)

22. Why is introspection essential for Java Beans technology?

Introspection can be defined as the technique of obtaining information about bean properties, events and methods. Basically introspection means analysis of bean capabilities. This is an essential feature of the Java Beans API because it allows another application, such as a design tool, to obtain information about a component.Without introspection, the Java Beans technology could not operate.

23. What is the difference between a simple property and an indexed property?
Simple properties refer to a property that can have only a single value. Simple properties are retrieved and specified using the get and set methods respectively.

An indexed property is a property that holds multiple values, typically in an array or a list. Indexed properties allow access to individual elements as well as the entire collection.

24. What are the two main components used to define a simple property in a Java Bean. Briefly explain their roles.
Simple properties are retrieved and specified using the get and set methods respectively. A read/write property has both of these methods to access its values. The get method used to read the value of the property. The set method that sets the value of the property.

25. What are the key differences between bound properties and constrained properties in Java Beans?
A bound property notifies registered listeners when its value changes. This allows other objects to react to the change. Bound Properties are always registered with an external event listener.
A constrained property generates an event when an attempt is made to change it value. The event is sent to previously registered objects. Those objects have the ability to veto the proposed change.

26. What is persistence in JavaBeans?
Persistence is the ability to save the current state of a Bean, including the values of a Bean's properties and instance variables, to nonvolatile storage and to retrieve them at a later time.

27. What are customizers?
A Bean developer can provide a customizer that helps another developer configure the Bean.
A customizer can provide a step-by-step guide through the process that must be followed to use the component in a specific context. Online documentation can also be provided.
A Bean developer has great flexibility to develop a customizer that can differentiate his or her product in the marketplace

Long Answer Questions

1. With an example explain how enumeration values are used to control a switch statement.

An enumeration value can also be used to control a switch statement. All of the case statements must use constants from the same enum as that used by the switch expression. There is no need to qualify the constants in the case statements with their enum type name.

Example:

```
// An enumeration of Colours
enum Colors {
        White, Black, Red
}
class Demo {
        public static void main(String args[])
{
                //enum value
                Colors c;
                c = Colors.Red;
                // Use an enum to control a switch statement.
                switch(c) {
                        case White:
                                System.out.println("This is white.");
                                break;
```

```
                    case Black:
                        System.out.println("This is black.");
                        break;
                    case Red:
                        System.out.println("This is red.");
                        break;
}}}
```
Output: This is red.

2. Demonstrate the usage of the valueOf() and values() methods with an example.

Values(): This method returns an array that contains a list of the enumeration constants. values() returns the values in the enumeration and stores them in an array. We can process the array with a foreach loop.
Syntax: public static *enum-type[ ] values( )*

ValueOf(): This method returns the enumeration constant whose value corresponds to the string passed in *str*. It takes a single parameter of the constant name to retrieve and returns the constant from the enumeration, if it exists.
Syntax: public static enum-type valueOf(String str)

Example:
```
enum Colors {
      White, Black, Red, Green
}
class Demo {
      public static void main(String args[]) {
            Colors cl;
            cl = Colors.valueOf("Red");               //use valueOf()
            System.out.println("cl contains: "+cl);
            System.out.println("Here are all Color constants:");
            Colors colors[] = Colors.values();     //use values()
            for(Colors c : colors) {
                  System.out.println(c);
                  System.out.println();
}}}
Output:           cl contains: Red
                  Here are all Color constants:
                  White
                  Black
                  Red
                  Green
```

3. What does ordinal(), compareTo() and equals() method do in Enum? Give an example.

ordinal(): Used to obtain a value that indicates an enumeration constant's position in the list of constants. Ordinal values begin at zero.
Syntax: final int ordinal()

compareTo(): It compares the ordinal value of two constants of the same enumeration. It returns 0 if they are equal, returns a negative integer if the current enum constant comes before the specified enum constant and returns a positive integer if the current enum constant comes after the specified enum constant.
Syntax: final int compareTo(enum-type e)

equals(): *equals()* checks if two enum constants are equal. It returns true if they are equal; otherwise returns false.
Syntax: boolean equals(Object other)

Example:

```
enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
public class Example {
public static void main(String[] args) {
   Day day1 = Day.MONDAY;
   Day day2 = Day.WEDNESDAY;
   // ordinal() example
   System.out.println("Ordinal of" +day1+ "is: " +day1.ordinal());
   // compareTo() example
   System.out.println(day1+ "compared to"+day2+"returns:"+day1.compareTo(day2));
   // equals() example
   System.out.println(day1+ "equals" +day2+ ":" +day1.equals(day2));
 }
}
```

Output: Ordinal of MONDAY is: 1
MONDAY compared to WEDNESDAY returns: -2
MONDAY equals WEDNESDAY: false

4. Java Enumerations Are Class Types. Explain with an example.

• Java enumeration is a class type. The fact that enum defines a class gives powers to the Java enumeration that enumerations in other languages simply do not have. For example, you can give them constructors, add instance variables and methods, and even implement interfaces.

• Each enumeration constant is an object of its enumeration type. Thus, when you define a constructor for an enum, the constructor is called when each enumeration constant is created. For example,

```
// Use an enum constructor, instance variable, and method.
enum Apple {
      Jonathan(10), RedDel(12), Winesap(15)
      private int price;      // price of each apple
      // Constructor
      Apple(int p) { price = p; }
      int getPrice() { return price; }
}
class Demo {
public static void main(String args[])  {
      Apple ap;
      // Display price of Winesap.
      System.out.println("Winesap costs"+Apple.Winesap.getPrice()+" cents.\n");
      // Display all apples and prices.
      System.out.println("All apple prices:");
      for(Apple a : Apple.values())
            System.out.println(a+ " costs " +a.getPrice()+ " cents.");
      }
}
```

Ouput: Winesap costs 15 cents.
All apple prices:
Jonathan costs 10 cents.
RedDel costs 12 cents.
Winesap costs 15 cents.

5. Explain with example how Enumerations can Inherit Enum?

All enumerations automatically inherit one: java.lang.Enum. This class defines several methods that are available for use by all enumerations. Some of them are: oridinal(), compareTo(), equals().

Refer Q.No. 3 for method definitions and example

6. Describe the Wrapper classes available for primitive types.

i. Character: Character is a wrapper around a char. The constructor for Character is *Character(char ch),* here ch is a character variable whose values will be wrapped to character object by the wrapper class.

• To obtain the char value contained in a Character object, call charValue( ), shown here:

char charValue( )

It returns the encapsulated character.

ii. Boolean: Boolean is a wrapper around boolean values. It defines these constructors:

Boolean(boolean boolValue)
Boolean(String boolString)

In the first version, boolValue must be either true or false. In the second version, if boolString contains the string "true" (in uppercase or lowercase), then the new Boolean object will be true. Otherwise, it will be false.

• To obtain a boolean value from a Boolean object, use booleanValue( ), shown here:

boolean booleanValue( )

It returns the boolean equivalent of the invoking object.

iii. Numeric Type Wrappers: The most commonly used type wrappers are those that represent numeric values. These are Byte, Short, Integer, Long, Float, and Double. All of the numeric type wrappers inherit the abstract class Number.

Number declares methods that return the value of an object in each of the different number formats. These methods are shown here:

1. byte byteValue( ) 2. double doubleValue( ) 3. float floatValue( )
4. int intValue( ) 5. long longValue( ) 6. short shortValue( )

doubleValue( ) returns the value of an object as a double, floatValue( ) returns the value as a float, and so on.

• All of the numeric type wrappers define constructors that allow an object to be constructed from a given value, or a string representation of that value. For example, here are the constructors defined for Integer: Integer(int num) Integer(String str)

7. What is Autoboxing And Unboxing? Explain with an example.

Autoboxing is the process by which a primitive type is automatically encapsulated (boxed) into its equivalent type wrapper whenever an object of that type is needed. There is no need to explicitly construct an object.

Auto-unboxing is the process by which the value of a boxed object is automatically extracted (unboxed) from a type wrapper when its value is needed. There is no need to call a method such as intValue( ) or doubleValue( ).

They are useful in removing the difficulty of manually boxing and unboxing values in several algorithms. It also helps prevent errors.

Example:

```
class AutoBox {
  public static void main(String args[]) {
        Integer iOb = 100;              // autobox an int
        int i = iOb;                    // auto-unbox
        System.out.println(i + " " + iOb); // displays 100 100
  } }
```

8. With example explain the steps involved to obtain annotation at run time using reflection.

• Reflection is the feature that enables information about a class to be obtained at run time. The reflection API is contained in the java.lang.reflect package.
• The first step to using reflection is to obtain a Class object that represents the class whose annotations you want to obtain. We can use getClass(), which is a method defined by Object.
• If you want to obtain the annotations associated with a specific item declared within a class, you must first obtain an object that represents that item.

Example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// An annotation type declaration.
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
        String str();
        int val();
}
class Meta {
        // Annotate a method.
        @MyAnno(str = "Example", val = 100)
        public static void myMeth() {
                Meta ob = new Meta();
                // Obtain the annotation for this method and display the values of the members.
                try {
                        //get a Class object that represents this class.
                        Class c = ob.getClass();
                        // get a Method object that represents this method.
                        Method m = c.getMethod("myMeth");
                        //get the annotation for this class.
                        MyAnno anno = m.getAnnotation(MyAnno.class);
                        //display the values.
                        System.out.println(anno.str() + " " + anno.val());
                } catch (NoSuchMethodException exc) {
                        System.out.println("Method Not Found.");
                 }
        }
        public static void main(String args[]) {
                myMeth();
}}
```

Output: Example 100

9. What Are Annotations? What are the three retention policies defined by the RetentionPolicy enumeration in Java?

Annotations in Java are a form of metadata that provide data about a program but are not part of the program itself. Annotations have no direct effect on the operation of the code they annotate. They can be used to provide information to the compiler, to be processed by tools, or to be available at runtime through reflection.

A retention policy determines at what point an annotation is discarded. Java defines three such policies, which are encapsulated within the **java.lang.annotation.RetentionPolicy** enumeration. They are **SOURCE**, **CLASS**, and **RUNTIME**.

• An annotation with a retention policy of **SOURCE** is retained only in the source file and is discarded during compilation.

• An annotation with a retention policy of **CLASS** is stored in the **.class** file during compilation. However, it is not available through the JVM during run time.

• An annotation with a retention policy of **RUNTIME** is stored in the **.class** file during compilation and is available through the JVM during run time. Thus, RUNTIME retention offers the greatest annotation persistence.

10. Write an example program to illustrate reflection.

Refer Q.No. 8

11. Explain any four Built-in annotations.

i. **@Override** : Used to indicate that a method overrides a superclass method. It ensures that the method is actually overriding a method in the superclass.

Example:
```
class Parent {
        public void display() {
                System.out.println("Parent's display method");
         }
}
class Child extends Parent {
         @Override
         public void display() {
                System.out.println("Child's display method");
         }
}
```

ii. **@Deprecated** : Marks a program element (class, method, etc.) as deprecated, meaning that it is no longer recommended for use. It serves as a warning to developers that the annotated element may be removed or modified in future versions.

Example:
```
class OldWay {
        @Deprecated
         public void oldMethod() {
                // Old implementation
}}
```

iii. **@Retention** : Specifies the retention policy for the annotated annotation type itself.

Example:
```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
        // Annotation elements declaration
}
```

iv. **@Documented** : It is a marker interface that tells a tool that an annotation is to be documented. It does not affect the runtime behavior of the annotated elements; rather, it affects the generated documentation.

Example:
```
import java.lang.annotation.Documented;
@Documented
@interface MyAnnotation {
        // Annotation elements declaration
}
```

12. How do you specify a default value for an annotation member? Explain with suitable example.

You can give annotation members default values that will be used if no value is specified when the annotation is applied

type member() default value;

Here, value must be of a type compatible with type. Example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// An annotation type declaration that includes defaults.
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
        String str() default “Testing”;
        int val() default 9000;
}
class Meta {
        // Annotate a method using default values
        @MyAnno()
        public static void myMeth() {
                Meta ob = new Meta();
                // Obtain the annotation for this method and display the values of the members.
                try {
                        Class c = ob.getClass();
                        Method m = c.getMethod("myMeth");
                        MyAnno anno = m.getAnnotation(MyAnno.class);
                        System.out.println(anno.str() + " " + anno.val());
                } catch (NoSuchMethodException exc) {
                        System.out.println("Method Not Found.");
                 }
        }
        public static void main(String args[]) {
                myMeth();
}}
```

Output: Testing 9000

13. Write an example of defining a marker annotation.

Here is an example that uses a marker annotation. Because a marker interface contains no members, simply determining whether it is present or absent is sufficient.

```
        import java.lang.annotation.*;
        import java.lang.reflect.*;
        // A marker annotation.
        @Retention(RetentionPolicy.RUNTIME)
        @interface MyMarker { }
        class Marker {
                // Annotate a method using a marker. Notice that no ( ) is needed.
                @MyMarker
                public static void myMeth() {
                        Marker ob = new Marker();
                        try {
                                Method m = ob.getClass().getMethod("myMeth");
                                // Determine if the annotation is present.
                                if(m.isAnnotationPresent(MyMarker.class))
                                        System.out.println("MyMarker is present.");
```

```
                } catch (NoSuchMethodException exc) {
                        System.out.println("Method Not Found.");
                }
        }
        public static void main(String args[]) {
                myMeth();
        }
}
```

Output: MyMarker is present

14. Write an example of defining a single-member annotation.

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// A single-member annotation.
@Retention(RetentionPolicy.RUNTIME)
@interface MySingle {
        int value(); // this variable name must be value
}
class Single {
        // Annotate a method using a single-member annotation.
        @MySingle(100)
        public static void myMeth() {
                Single ob = new Single();
                try {
                        Method m = ob.getClass().getMethod("myMeth");
                        MySingle anno = m.getAnnotation(MySingle.class);
                        System.out.println(anno.value()); // displays 100
                } catch (NoSuchMethodException exc) {
                        System.out.println("Method Not Found.");
                }
        }
        public static void main(String args[]) {
                myMeth();
        }}
```

Output: 100

15. How Can You Retrieve all Annotations that have RUNTIME retention by use of reflection? Explain with an example.

You can obtain all annotations that have RUNTIME retention that are associated with an item by calling getAnnotations() on that item.

Syntax: Annotation[ ] getAnnotations( )

It returns an array of the annotations. getAnnotations( ) can be called on objects of type Class, Method, Constructor, and Field.

Example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
        String str();
        int val();
}
```

```
@Retention(RetentionPolicy.RUNTIME)
@interface What {
        String description();
}

@What(description = "An annotation test class")
@MyAnno(str = "Meta2", val = 99)
class Meta2 {
        @What(description = "An annotation test method")
        @MyAnno(str = "Testing", val = 100)
        public static void myMeth() {
                Meta2 ob = new Meta2();
                try {
                        Annotation annos[] = ob.getClass().getAnnotations();
                        // Display all annotations for Meta2.
                        System.out.println("All annotations for Meta2:");
                        for(Annotation a : annos)
                                System.out.println(a);
                                System.out.println();
                        // Display all annotations for myMeth.
                        Method m = ob.getClass( ).getMethod("myMeth");
                        annos = m.getAnnotations();
                        System.out.println("All annotations for myMeth:");
                        for(Annotation a : annos)
                                System.out.println(a);
                } catch (NoSuchMethodException exc) {
                        System.out.println("Method Not Found.");
        }}
        public static void main(String args[]) {
                myMeth();
        }}
Output:         All annotations for Meta2:
                @What(description=An annotation test class)
                @MyAnno(str=Meta2, val=99)
                All annotations for myMeth:
                @What(description=An annotation test method)
                @MyAnno(str=Testing, val=100)
```

16. Discuss the key advantages that Java Beans provide to component developers.

- The java beans possess the property of "Write once and run anywhere".
- Beans can work in different local platforms.
- Beans have the capability of capturing the events sent by other objects and vice versa enabling object communication.
- The properties, events and methods of the bean can be controlled by the application developer. (ex. Add new properties)
- Beans can be configured with the help of auxiliary software during design time.(no hassle at runtime)
- The configuration setting can be made persistent.(reused)
- Configuration setting of a bean can be saved in persistent storage and restored later.

17. What are the different properties of a Java Bean? Explain with examples.

There are two types of properties: simple and indexed.

Simple properties refer to a property that can have only a single value. It can be identified by the following design patterns, where N is the name of the property and T is its type:

```
public T getN( )
public void setN(T arg)
```

A read/write property has both of these methods to access its values. A read-only property has only a get method. A write-only property has only a set method.

Example:

```
private double height;
public double getHeight( ) {
        return height;
}
public void setHeight(double h) {
        height = h;
}
```

An ***indexed property*** is a property that holds multiple values, typically in an array or a list. It can be identified by the following design patterns, where N is the name of the property and T is its type:

```
public T getN(int index);
public void setN(int index, T value);
public T[ ] getN( );
public void setN(T values[ ]);
```

Example:

```
private double data[ ];
public double getData(int index) {     return data[index];      }
public void setData(int index, double value) {          data[index] = value;    }
public double[ ] getData( )  {          return data;      }
public void setData(double[ ] values)  {
        data = new double[values.length];
        System.arraycopy(values, 0, data, 0, values.length);
}
```

18. Write short note on the following

i. Bound and Constrained properties **ii. Persistance**

i. Bound and Constrained properties:

A bound property notifies registered listeners when its value changes. This allows other objects to react to the change. Bound Properties are always registered with an external event listener.

The event is of type PropertyChangeEvent and is sent to objects that previously registered an interest in receiving such notifications

bean with bound property - Event source
Bean implementing listener - Event target

A constrained property generates an event when an attempt is made to change it value. The event is sent to objects that previously registered an interest in receiving an such notification. Those objects have the ability to either accept or reject the change in the value of a constrained property.

ii. Persistance:
Persistence is the ability to save the current state of a Bean, including the values of a Bean's properties and instance variables, to nonvolatile storage and to retrieve them at a later time.

The object serialization capabilities provided by the Java class libraries are used to provide persistence for Beans. The easiest way to serialize a Bean is to have it implement the java.io.Serializable interface, which is simply a marker interface. Implementing java.io.Serializable makes serialization automatic. Your Bean need take no other action. Automatic serialization can also be inherited.

19. Write a note on simple properties in java Beans.

Refer Q.No. 17

20. Write a note on indexed properties in java Beans.

Refer Q.No. 17

21. What are the steps to be followed in creating a new Bean?

1. Create a directory for the new bean
   Create a directory/folder like C:\Beans

2. Create the java bean source file(s)

```
import java.awt.*;
public class MyBean extends Canvas  {
        public MyBean()  {
                setSize(70,50);
                setBackground(Color.green);
} }
```

3. Compile the source file(s)
   C:\Beans > javac MyBean.java

4. Create a manifest file
   Manifest-Version: 1.0
   Name: MyBean.class
   Java-Bean: true

5. Generate a JAR file
   C:\Beans >jar cfm MyBean.jar MyBean.mf MyBean.class

6. Start BDK
   Go to-> C:\bdk1_1\beans\beanbox
   Click on run.bat file. When we click on run.bat file the BDK software automatically started.

7. Load Jar file
   Go to Beanbox->File->Load jar. Here we have to select our created jar file when we click on ok, our bean(userdefined) MyBean appear in the ToolBox.

8. Test.
Select the MyBean from the ToolBox when we select that bean one + simple appear then drag that Bean in to the Beanbox.

22. Explain the purpose and usage of the following classes in the context of Java Beans:

Introspector
PropertyDescriptor
EventSetDescriptor
MethodDescriptor

Introspector: The Introspector class provides several static methods that support introspection. Of most interest is **getBeanInfo( )**. This method returns a **BeanInfo** object that can be used to obtain information about the Bean. The **getBeanInfo( )** method has several forms, including the one shown here: *static BeanInfo getBeanInfo(Class bean) throws IntrospectionException*. The returned object contains information about the Bean specified by bean.

PropertyDescriptor: The PropertyDescriptor class describes a Bean property. It supports several methods that manage and describe properties. For example, you can determine if a property is bound by calling **isBound( )**. To determine if a property is constrained, call **isConstrained( )**.

EventSetDescriptor: The EventSetDescriptor class represents a Bean event. It supports several methods that obtain the methods that a Bean uses to add or remove event listeners, and to otherwise manage events. For eg, to obtain the method used to add listeners, call **getAddListenerMethod( )**. To obtain the method used to remove listeners, call **getRemoveListenerMethod( ).**

MethodDescriptor: The MethodDescriptor class represents a Bean method. To obtain the name of the method, call **getName( )**.You can obtain information about the method by calling **getMethod( )**, shown here: *Method getMethod( )*. An object of type Method that describes the method is returned.

23. Write a note on EventDescriptor and PropertyDescriptor class.

Refer Q.No. 22